

요구사항에서 실제 구현까지

ChatPRD × Spec Kit × Codex

자연어를 곧바로 코드로 보내지 않고,
명세·설계·Task를 구현과 검증의 계약으로 연결했다.

3 Test Files

7 Tests PASS

요구사항 정의



시스템·로직 설계



AI 자동 코딩 + 검증

3단계 Toolchain을 실제 결과로 증명한다

1. 핵심 과제: 도구 기반 바이브 코딩 파이프라인 실습 및 발표

단순 채팅창 대화 방식의 코딩을 넘어, [요구사항 정의 → 설계 → AI 코딩]으로 이어지는 3단계 도구 연계(Toolchain)를 직접 구성해보고 그 프로세스를 발표해야 합니다.

단계	수행 내용	활용/조사 도구 예시
1. 요구사항 정의	프로젝트 요구사항(PRD / RFP)을 구조화하여 작성	Notion AI, ChatPRD, 요구사항 템플릿 툴
2. 시스템/로직 설계	작성된 PRD를 기반으로 아키텍처·FSM·데이터 흐름 설계	Mermaid.js, Eraser.io, PlantUML, Draw.io
3. AI 자동 코딩	설계 문서를 입력받아 코드를 구현하도록 도구 연계	Cursor, VS Code (GitHub Copilot / Claude Code 등)

...

- 발표 요구사항: 각자 선택한 도구들의 연계 워크플로우와 실제 동작 결과를 정리하여 공유.

1

ChatPRD

요구사항 정의

PRD

2

Spec Kit

시스템·로직 설계

Spec · Plan · Tasks

3

Codex

AI 자동 코딩

Code · Tests

바이브 코딩의 속도는 “계약”이 없을 때 흔들린다

빠르다

- 생성된다
- 일단 실행된다

하지만

- 요구사항이 변한다
- 예외가 누락된다
- 검증 기준이 흐려진다

속도보다 먼저 고정할 것

무엇을 만들 것인가 · 어떤 구조로 만들 것인가 · 무엇으로 완료를 판단할 것인가

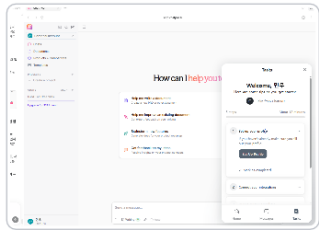
그래서 자연어 → PRD → 설계 산출물 → Task → 코드 → 테스트의 연속성을 만들었다.

불완전한 자연어 요구를 PRD로 발전시켰다

ChatPRD 실습 — 자연어 요구사항에서 PRD까지

IP 카메라 관리 시스템 요구를 동일한 흐름으로 검증

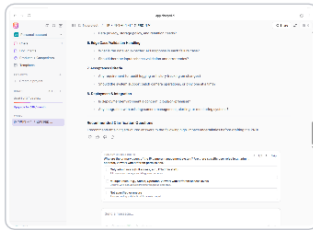
1 초기 요구 입력



카메라 CRUD · Ping ·
TCP Port Check

불완전한 자연어 요구

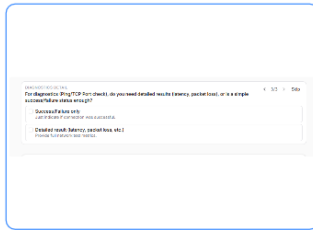
2 Gap Analysis



사용자 · 인증 · UI ·
Network · NFR · 배포

누락·모호성 자동 식별

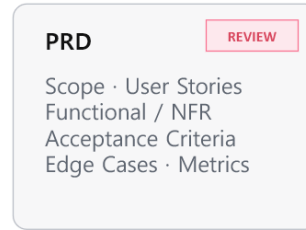
3 Clarification



Admin only
No authentication
Detailed result

질문으로 제약조건 확정

4 PRD + Review



모호성 · 충돌 · 임의
가정 · 성능조건 검토

구조화 후 다시 품질 검증

결론 불완전한 자연어 요구를 질문으로 구조화된 PRD까지 발전시켰다

초기 요청

카메라 CRUD
Ping · TCP Port Check

ChatPRD가 보완한 것

사용자 · 역할 · UI
예외 · NFR · 메트릭

질문을 통해 “문서로 검토 가능한 요구 사항”이 됐다.

질문은 선명했지만, 최종 일관성은 사람이 확인해야 했다

질문은 요구를 선명하게 했지만, 일관성은 보장하지 못했다

실제 Clarification 선택과 Review 수정안을 나란히 비교

Clarification에서 확정

PRIMARY USERS & ROLES
Who are the primary users of the IP camera management system? Are there specific user roles (e.g., admin, operator, viewer) with different permissions?

☐ Only admin users (full access, e.g., IT/admin staff)
All users can manage and diagnose cameras.

☒ Multiple roles (e.g., Admin, Operator, Viewer) with different access levels
Control who can add/delete/diagnose cameras. ✓ Admin only

API SECURITY
Should API access be authenticated? If yes, which approach is preferred?

☐ No authentication
Anyone with access can use the API.

☒ API key authentication
Each user, app, or client needs a key. ✓ No authentication

DIAGNOSTICS DETAIL
For diagnostics (Ping/TCP Port check), do you need detailed results (latency, packet loss), or is a simple success/failure status enough?

☐ Success/Failure only
Just indicate if connection was successful.

☒ Detailed result (latency, packet loss, etc.)
Provide full network test metrics. ✓ Detailed result

PRD 생성 후 Review

구조화된 산출물

Scope · User Stories · Functional Requirements · NFR
Acceptance Criteria · Edge Cases · Success Metrics

그러나 Review 수정안에서

확정 요구
No authentication

Review 수정안
authenticated admin
users

이전 결정과 충돌하는 표현이 다시 생성됨

최종 Human Review 필요

강점

Gap Analysis와 Clarification으로 누락을 드러냄

한계

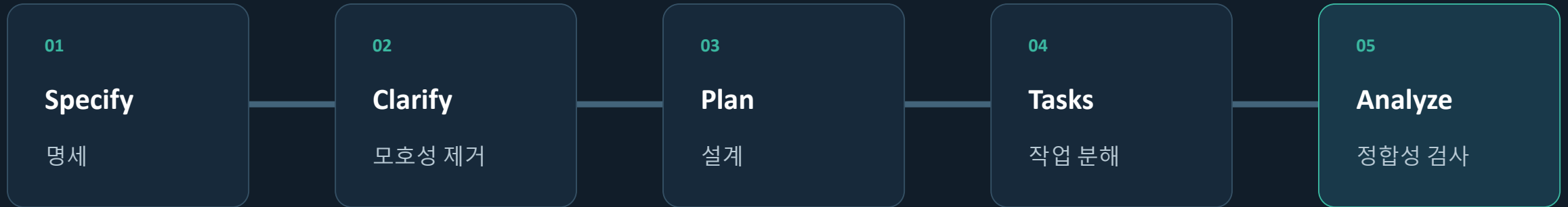
Review 중 Network authentication 조건이 흔들림

판단

PRD 생성 완료 ≠ 요구사항 확정 완료

Human Review가 마지막 품질 게이트

요구사항을 설계·Task·검증으로 연결했다



산출물

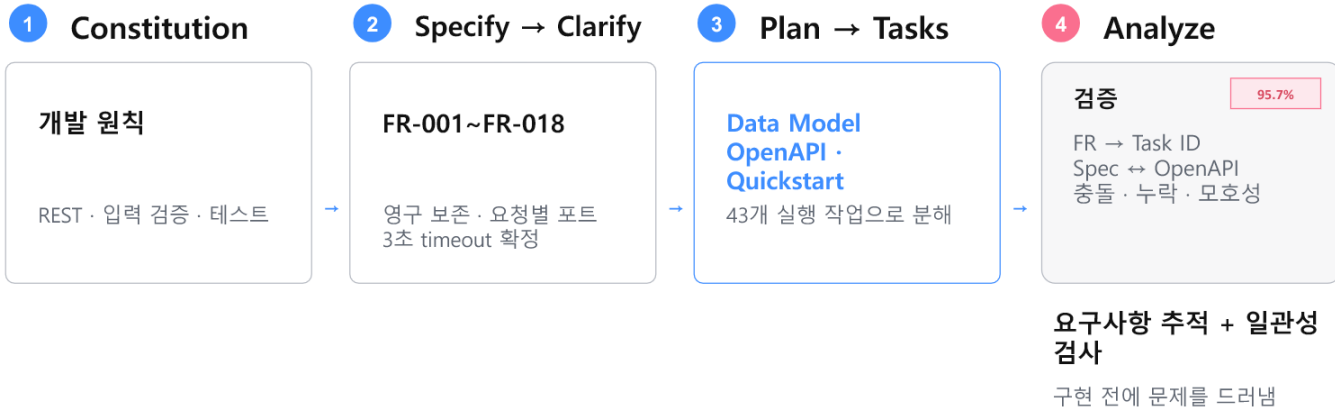
spec.md · plan.md · data-model.md · openapi.yaml · tasks.md

각 문서는 다음 단계의 입력이 되고, Analyze가 구현 전 충돌과 누락을 다시 확인한다.

기능 요구 FR-001~FR-018과 모호성을 먼저 고정했다

Spec Kit 실습 — 명세를 설계·Task·검증으로 연결

Constitution → Specify → Clarify → Plan → Tasks → Analyze



핵심 명세를 설계·계약·Task까지 연결하고, 구현 전 교차검증했다

Specify

Camera CRUD + 진단 범위를
검토 가능한 명세로 전환

Clarify

인증·입력·예외 조건의
모호성을 질문으로 제거

구현보다 먼저 “무엇이 맞는가”를 합의

설계는 코드 모양보다 입력과 출력의 경계를 그렸다

Data Model

Camera entity

OpenAPI

CRUD contract

Quickstart

실행·검증 절차

Project Structure

src / tests / specs

Plan의 역할

요구사항을 구현 가능한 기술 계약으로 번역

TypeScript · Fastify · SQLite

Camera CRUD MVP

설계 산출물은 Codex가 추측해야 할 여지를 줄이는 입력 컨텍스트가 됐다.

43개 Task로 분해하고 구현 전 정합성을 확인했다

Analyze는 요구사항이 Task까지 연결됐는지 증명한다

FR-001~FR-018과 성공 기준을 43개 구현 작업에 대조

Coverage Summary

Coverage Summary			
Requirement	Has Task?	Task IDs	Notes
FR-001	Yes	T014, T017-T020	CRUD 생성
FR-016	Yes	T006, T016	영구 보존
FR-017	Yes	T029, T036	요청별 포트
FR-018	Yes	T029-T034	3초 timeout
SC-002	Yes	T021-T028	초기 검증
SC-003	Yes	T029-T037	진단 상태
SC-004	Partial	T041	전용 성능 테스트 없음
SC-006	Yes	T041-T043	전체 자동화 테스트

요구사항 추적 결과

95.7% 요구사항 커버리지

22/23 항목이 Task와 연결
기능 요구사항 18개 · 총 작업 43개

FR → Task ID 매핑

FR-016
T006 · T016
영구 보존

→

FR-018
T029-T034
3초 timeout

SC-004만 Partial — 전용 성능 테스트 작업이 없음

요구사항이 실제 구현 Task까지 연결되는지 추적

Analyze는 구현 전에 충돌과 누락을 드러냈다

Spec · Plan · Tasks · OpenAPI를 교차검증한 결과

자동 검출된 이슈

ID	Category	Severity	LocationID	Summary	Recommendation
1	Requirements	INFO	* spec.req.01 / openapi.req.01	FR-012는 모든 성공 응답을 JSON으로 요구하지만, 삭제 케이스는 204 No Content를 반환합니다.	204를 제외하고는 FR-012를 수정하거나 삭제 응답을 204 JSON으로 통일하세요.
2	Requirements	INFO	* spec.req.01 / openapi.req.01	Camera Name의 최대 길이가 정의되지 않았고, OpenAPI는 255로 정의되어 있습니다.	최대 길이를 설정하고, 예제, 스키마, 테스트에 동기화해서 반영하세요.
3	Coverage	MEDIUM	* plan.req.01 / task.req.01	SC-004의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
4	Coverage	MEDIUM	* plan.req.01 / task.req.01	SC-004의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
5	Requirements	MEDIUM	* spec.req.01 / openapi.req.01	Camera Name의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
6	Requirements	MEDIUM	* spec.req.01 / openapi.req.01	Camera Name의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
7	Requirements	MEDIUM	* spec.req.01 / openapi.req.01	Camera Name의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
8	Requirements	MEDIUM	* spec.req.01 / openapi.req.01	Camera Name의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
9	Requirements	MEDIUM	* spec.req.01 / openapi.req.01	Camera Name의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.
10	Requirements	MEDIUM	* spec.req.01 / openapi.req.01	Camera Name의 '영구'는 '이름' 기준에 따른 성능 기준 작업에 포함되지 않습니다.	성능 측정 기준을 '이름' 대신 '작업'으로 추가하고, 작업 범위를 정의하세요.

Critical 0 · Constitution 위반 0
하지만 HIGH 2건은 구현 전에 정리 필요

구현 전 해결해야 할 3가지

HIGH · 계약 충돌

FR-012: 모든 성공 응답 JSON
↔ DELETE 204 No Content

입력 제약과 검증 작업도 확인

HIGH
Camera Name
최대 길이 미정

MEDIUM
SC-004
성능 테스트 누락

AI가 만든 산출물도 문서 간 교차검증이 필요

ChatPRD는 PRD 구체화 · Spec Kit은 설계·Task 연결과 일관성 검증

T001-T043

요구사항 추적

충돌·누락 탐지

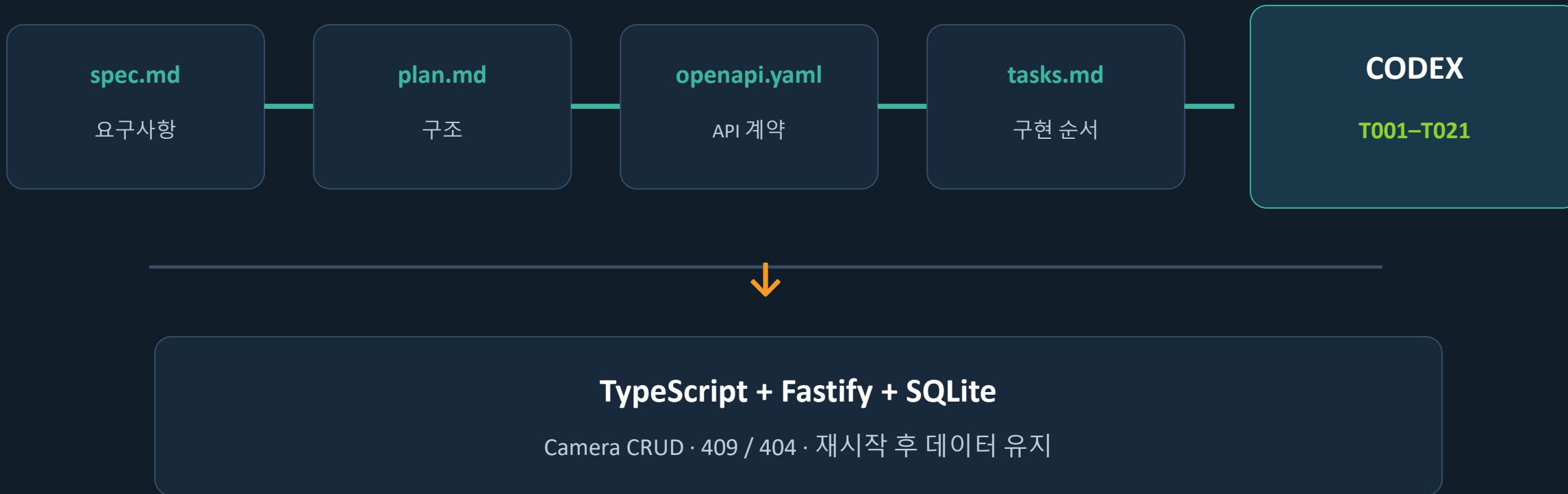
구현 전 수정

요구사항이 Task와 테스트까지 연결되는지 추적했다

요구사항	설계 계약	Task	검증
Camera CRUD	OpenAPI / Camera model	T001-T021	Contract + Lifecycle
Persistence	SQLite / WAL	T001-T021	Restart persistence
Ping / TCP	Diagnostics contract	T022-T043	후속 범위

완료 판정은 “코드가 생겼다”가 아니라 요구사항 → 계약 → Task → 테스트가 이어졌는가로 판단했다.

Spec Kit 산출물을 Codex의 구현 입력으로 사용했다



자연어가 아니라 검토된 문서 묶음이 구현 컨텍스트가 됐다.

MVP 범위 T001-T021을 실제 코드로 구현했다

```
spec Kit MVP 카메라 CRUD 구현
11m 동안 작업 >

요청 범위인 Setup, Foundational, US1 Camera CRUD(T001-T021)를 구현하고 체크했습니다.

구현 결과
• TypeScript + Fastify 프로젝트 구성
• SQLite 영속 저장소, WAL 모드, 트랜잭션 헬퍼
• Camera 및 공통 오류 타입-스키마
• 공통 JSON 오류 처리
• 애플리케이션 팩토리와 graceful shutdown
• Camera 생성-목록-상세-수정-삭제 API
• 중복 ID 409, 미등록 Camera 404
• 앱 재시작 후 SQLite 데이터 유지
• OpenAPI CRUD 요청-응답 예제 추가
• 테스트용 격리 DB 및 애플리케이션 fixture
• node_modules, 빌드 및 DB 산출물 제외 설정

완료 상태는 tasks.md에 반영했습니다.

검증
모두 통과했습니다.
• 테스트: 3개 파일, 7개 테스트 통과
• npm run typecheck
• npm run lint
• npm run build
• npm run format:check
• 의존성 설치 당시 보안 취약점: 0건

변경 파일
프로젝트 설정:
• package.json, package-lock.json
• tsconfig.json, vitest.config.ts
• eslint.config.js, .prettierrc.json, .gitignore

구현:
• src/config.ts
```

Runtime

TypeScript + Fastify

Storage

SQLite · WAL · Transaction

API

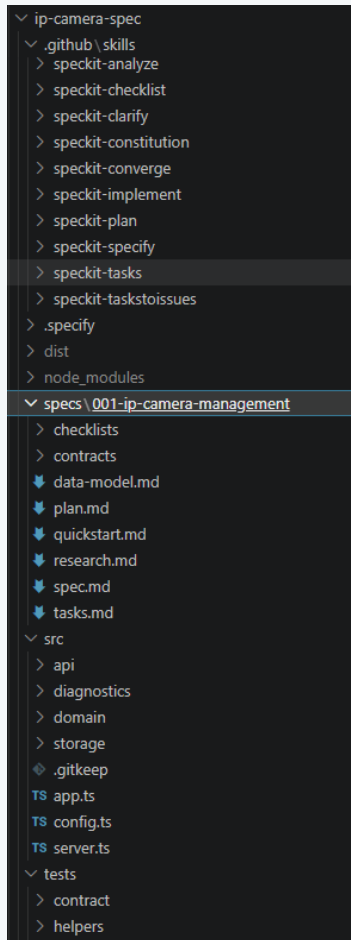
Create · List · Read · Update · Delete

Errors

Duplicate 409 · Missing 404

Setup · Foundational · US1 완료

설계 산출물과 코드·테스트가 한 프로젝트에 연결됐다



specs/

spec.md · plan.md
data-model.md · ope
napi.yaml
tasks.md

src/

api · domain · storag
e
app.ts · server.ts

tests/

contract · integration
setup · helpers

문서

→ 구현 →

검증

프로젝트 구조 자체가 Toolchain 연결의 증거

3개 테스트 파일 · 7개 테스트가 모두 통과했다

```
PS C:\Users\netvision\Desktop\netvision\netvision-study\practice\minwoo\ip-camera-spec> npm test

> ip-camera-management@1.0.0 test
> vitest run --configLoader runner

RUN v3.2.7 C:/Users/netvision/Desktop/netvision/netvision-study/practice/minwoo/ip-camera-spec

✓ tests/integration/camera-persistence.test.ts (1 test) 773ms
  ✓ camera persistence > preserves camera data after the application is restarted 771ms
✓ tests/contract/cameras-crud.test.ts (3 tests) 902ms
  ✓ camera create and read contract > creates, lists, and retrieves a camera 735ms
✓ tests/contract/cameras-lifecycle.test.ts (3 tests) 921ms
  ✓ camera update and delete contract > updates camera name and address while keeping its ID 733ms

Test Files  3 passed (3)
Tests       7 passed (7)
Start at    09:21:52
Duration    1.84s (transform 162ms, setup 40ms, collect 1.32s, tests 2.60s, environment 1ms, prepare 432ms)

❖ PS C:\Users\netvision\Desktop\netvision\netvision-study\practice\minwoo\ip-camera-spec> s
```

Contract

Lifecycle

Restart persistence

7 / 7 PASS

완료한 MVP와 후속 작업의 경계를 명확히 남겼다

완료 · T001-T021

Setup · Foundational
US1 Camera CRUD
SQLite persistence
409 / 404 오류 처리
Contract · Lifecycle · Integration tests

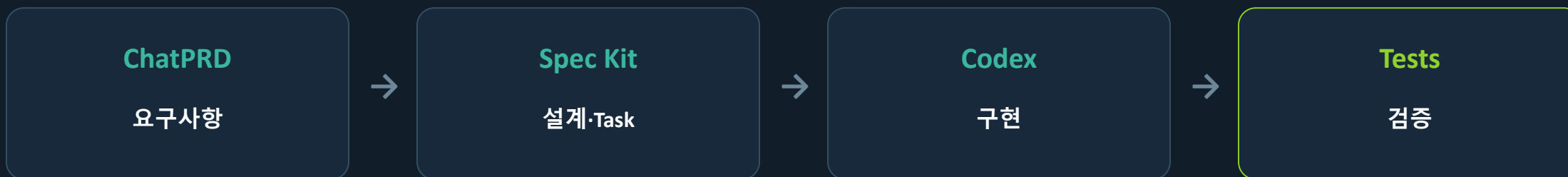
후속 · T022-T043

US2 Ping / TCP 진단
US3 입력·예외 검증 강화
Quickstart 전체 실행
OpenAPI conformance
Polish

이번 실습의 목적은 전체 기능 완성이 아니라 Toolchain의 실제 연결과 검증이었다.

CONCLUSION

도구를 연결하면 대화가 증거가 된다



요구사항 → 설계 산출물 → T001-T021 구현 → 7/7 PASS

다음 단계: T022-T043으로 Ping/TCP 진단과 검증 범위를 확장